ESP32 Connect

Estudo de Caso: Arquitetura e Visão de Produto

Desenvolvido por:
Rodrigo Costa de Araujo
Engenharia Elétrica - POLI-USP

---

1 O Problema: Exclusão no Ecossistema Educacional

Durante o curso técnico em Eletrônica, observei um gargalo educacional significativo: ferramentas populares para introdução à criação de aplicativos móveis integrados com hardware, como o MIT App Inventor, não oferecem suporte adequado para comunicação Bluetooth Low Energy (BLE) em dispositivos Apple (iOS).

Essa limitação técnica excluía os alunos usuários de iPhone das aulas práticas de robótica e IoT, enquanto os alunos com dispositivos Android conseguiam concluir as atividades normalmente. Havia uma clara necessidade de uma plataforma que democratizasse o acesso ao controle de microcontroladores para o ecossistema iOS.

2 A Solução e Visão de Produto

Para suprir essa demanda, foi projetado e desenvolvido o **ESP32 Connect**, um aplicativo nativo para iOS. O escopo original evoluiu rapidamente de uma ferramenta educacional para um *Hub* de automação corporativa e residencial.

O sistema é capaz de parear e gerenciar múltiplos microcontroladores ESP32 de forma simultânea, controlando desde I/O digital básico e PWM, até atuadores complexos (servomotores) e leituras contínuas de sensores (DHT11 para termometria e HC-SR04 para telemetria de distância).

Figura 1: Interface de controle do ESP32 Connect em execução no iOS.

Roadmap de Lançamento: Devido à robustez alcançada pela arquitetura, o aplicativo encontra-se em fase final de refinamento para ser submetido e publicado oficialmente na **Apple App Store**, transitando de uma solução de nicho para um produto global de prateleira.

3 Arquitetura do Sistema e Decisões de Engenharia

3.1 Front-end e Camada de Rede (iOS / Swift)

O desenvolvimento nativo em **Swift** e **SwiftUI** garantiu performance otimizada e acesso de baixo nível às APIs de conectividade da Apple.

- **Conectividade Híbrida:** Utilização do *CoreBluetooth* para conexões BLE de baixo consumo e do framework *Network* para conexões TCP/IP em redes Wi-Fi locais via mDNS (Bonjour).
- **Resiliência de Dados (Buffer TCP):** Um dos maiores desafios em sistemas IoT distribuídos é a fragmentação de pacotes na rede. Para mitigar isso, foi implementado um sistema de buffer no `WiFiManager` do iOS. O algoritmo acumula pacotes e só realiza o *parsing* ao identificar o delimitador de fim de instrução, prevenindo falhas críticas no painel de controle do usuário.

3.2 Firmware Embarcado (C/C++ e FreeRTOS)

O firmware do ESP32 foi projetado para atuar como um servidor autônomo, não dependendo exclusivamente do status da conexão com o smartphone.

- **Processamento Dual-Core:** Para evitar gargalos e latência, as tarefas foram divididas. Uma *Task* do FreeRTOS foi alocada exclusivamente no Core 0 para lidar com chamadas HTTPs (sincronização de telemetria), garantindo que o *loop* principal de controle de hardware no Core 1 opere em tempo real sem interrupções.
- **Engine de Regras de Borda (*Edge Computing*):** A lógica condicional de automação é salva diretamente na memória não-volátil (NVS) do microcontrolador. Caso uma regra estipule que um relé deve ser acionado se a temperatura exceder 30°C, o próprio ESP32 processa essa variável e executa a ação, mesmo estando desconectado do aplicativo.

4 Integração Cloud e Escalonamento

O ecossistema conta com integração a serviços em nuvem (Supabase/PostgreSQL) via API REST. Os dados coletados pelos microcontroladores (como temperatura, umidade e distância) são empurrados (*pushed*) continuamente para a nuvem, criando uma base de dados estruturada que abre portas para futuras análises de *Machine Learning* e desenvolvimento de *Dashboards* preditivos para manutenção industrial.

5 Conclusão

O **ESP32 Connect** valida a capacidade de projetar uma arquitetura de sistemas completa (*End-to-End*): desde o *bare metal* do microcontrolador em C++, passando pela modelagem de comunicação em rede (Bluetooth e TCP), até a entrega de uma interface mobile (UX/UI) comercializável.